Autonomous and Mobile Robotics

Localization: Odometric Localization and the Extended Kalman Filter

Course of Prof. Giuseppe Oriolo — Sapienza Università di Roma

November 5, 2026

Contents

1 Why localization is needed *(odometry deck, slide 2)* **2**

2 Odometric localization: integrating the kinematic model *(odometry deck, slides 3–6)* **2**
- 2.1 Setting: constant inputs over a sampling interval . . . 2
- 2.2 Three integration schemes . . . 3
- 2.3 How different are they? Geometry and error order . . . 3

3 Reconstructing the inputs from proprioceptive sensors *(odometry deck, slide 7)* **5**

4 Dead reckoning and odometric drift *(odometry deck, slides 8–9)* **6**

5 From dead reckoning to probabilistic localization *(EKF deck, slide 2)* **8**

6 Basic concepts: mean, covariance, Gaussians *(EKF deck, slides 3–4)* **8**

7 Kalman filter without noise *(EKF deck, slides 5–9)* **9**
- 7.1 Recursive observer: prediction and correction . . . 9
- 7.2 The correction as a constrained optimization . . . 9
- 7.3 Why this naive observer is not enough . . . 10

8 Kalman filter with process noise only *(EKF deck, slides 10–15)* **11**
- 8.1 Prediction . . . 11
- 8.2 State correction: Mahalanobis distance and weighted pseudoinverse . . . 11
- 8.3 Covariance correction and its flaw . . . 12

9 Kalman filter: the full version *(EKF deck, slides 16–21)* **12**
- 9.1 State correction with a noisy measurement . . . 13
- 9.2 Properties of the KF . . . 14

10 The Extended Kalman Filter *(EKF deck, slides 22–24)* **14**

11 EKF localization of a unicycle with range-and-bearing landmarks *(book, Sect. 7.7.2)* **16**
- 11.1 Motion model . . . 16
- 11.2 Measurement model . . . 16
- 11.3 The EKF equations for this problem . . . 17

12 Summary **19**
12.1 Comparison of methods 19
12.2 Key take-aways 19

Conventions. Slide ranges in the section headers refer to the *Odometric Localization* deck (Sections 1–4) or to the *Extended Kalman Filter* deck (Sections 5–10). Section 11 is the application worked out in the companion book chapter (Sect. 7.7.2 there), merged at the point where the EKF has just been introduced. The robot is a unicycle with configuration $\boldsymbol{q} = (x, y, \theta)$, kinematic inputs v (driving velocity) and ω (steering velocity), and sampling interval $T_s = t_{k+1} - t_k$.

1 Why localization is needed *(odometry deck, slide 2)*

Planning and, above all, feedback control need the robot configuration $\boldsymbol{q}$ at every time instant: a trajectory-tracking law computes its commands from the *error* between the desired and the actual configuration, so it cannot even be written down without an estimate of $\boldsymbol{q}$.

For a robot manipulator this is easy: joint encoders measure the joint variables, which *are* the configuration. A wheeled mobile robot (WMR) is different. Its incremental encoders measure only how much each wheel has rotated, *not* where the vehicle is with respect to a fixed world frame.

Definition 1.1 (Localization)**.** *Localization* is a procedure that estimates, typically in real time, the configuration $\boldsymbol{q}(t)$ of a robot with respect to a fixed world frame from the available sensor data.

Intuition

A joint encoder is like a GPS: it tells you directly *where* you are. A wheel encoder is like a pedometer: it tells you how many steps you took, from which you can *reconstruct* where you are only if you know where you started and how long and in which direction each step was. The whole lecture follows this analogy: first we add up the steps (odometry), then we learn to correct the sum by looking around (Kalman filtering with exteroceptive sensors).

2 Odometric localization: integrating the kinematic model *(odometry deck, slides 3–6)*

2.1 Setting: constant inputs over a sampling interval

Recall the unicycle kinematic model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{1}$$

In a digital implementation the velocity inputs are held constant over each sampling interval: $v(t) = v_k$, $\omega(t) = \omega_k$ for $t \in [t_k, t_{k+1})$. Then $\theta(t) = \theta_k + \omega_k(t - t_k)$, so the heading changes at a constant rate, and the robot moves along an arc of circle of radius $|v_k/\omega_k|$ (a line segment if $\omega_k = 0$).

Problem. Given $\boldsymbol{q}_k = (x_k, y_k, \theta_k)$ and (v_k, ω_k), compute $\boldsymbol{q}_{k+1}$ by integrating (1) over $[t_k, t_{k+1}]$.

Intuition

Constant "gas" and constant "steering wheel" means driving on a circle. Over one sampling interval the robot therefore covers a short arc; the only question is how faithfully we want to reproduce that arc with a formula.

2.2 Three integration schemes

Algorithm 2.1 (Euler integration).

$$\begin{aligned} x_{k+1} &= x_k + v_k T_s \cos\theta_k, \\ y_{k+1} &= y_k + v_k T_s \sin\theta_k, \\ \theta_{k+1} &= \theta_k + \omega_k T_s. \end{aligned} \tag{2}$$

x_{k+1} and y_{k+1} are *approximate* (computed as if the orientation stayed equal to θ_k throughout the interval); θ_{k+1} is *exact*.

Algorithm 2.2 (Second-order Runge–Kutta integration).

$$\begin{aligned} x_{k+1} &= x_k + v_k T_s \cos\Big(\theta_k + \frac{\omega_k T_s}{2}\Big), \\ y_{k+1} &= y_k + v_k T_s \sin\Big(\theta_k + \frac{\omega_k T_s}{2}\Big), \\ \theta_{k+1} &= \theta_k + \omega_k T_s. \end{aligned} \tag{3}$$

The *average orientation* over $[t_k, t_{k+1}]$ is used; x_{k+1}, y_{k+1} are still approximate but more accurate.

Proposition 2.3 (Exact integration). *Under constant v_k, ω_k with $\omega_k \neq 0$,*

$$\begin{aligned} x_{k+1} &= x_k + \frac{v_k}{\omega_k}\big(\sin\theta_{k+1} - \sin\theta_k\big), \\ y_{k+1} &= y_k - \frac{v_k}{\omega_k}\big(\cos\theta_{k+1} - \cos\theta_k\big), \\ \theta_{k+1} &= \theta_k + \omega_k T_s. \end{aligned} \tag{4}$$

For $\omega_k \to 0$ the first two formulas tend to the Euler (and Runge–Kutta) ones, so they remain defined at $\omega_k = 0$.

Proof. Since $\theta(t) = \theta_k + \omega_k(t - t_k)$ we have $dt = d\theta/\omega_k$, hence

$$x_{k+1} - x_k = \int_{t_k}^{t_{k+1}} v_k \cos\theta \, dt = \frac{v_k}{\omega_k} \int_{\theta_k}^{\theta_{k+1}} \cos\theta \, d\theta = \frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k),$$

and similarly for y. For the limit, $\sin(\theta_k + \omega_k T_s) - \sin\theta_k = \omega_k T_s \cos\theta_k + O(\omega_k^2)$, so $x_{k+1} - x_k \to v_k T_s \cos\theta_k$ (and likewise for y) as $\omega_k \to 0$. □

Intuition

For $\omega_k \approx 0$ the exact formula is a "0/0" expression: numerically it divides a tiny number by a tiny number. In an implementation one therefore adds a conditional instruction: if $|\omega_k|$ is below a small threshold use the straight-line (Euler/Runge–Kutta) update, otherwise the closed form.

2.3 How different are they? Geometry and error order

Figure 1 compares the three reconstructions of $\boldsymbol{q}_{k+1}$ on a single arc.

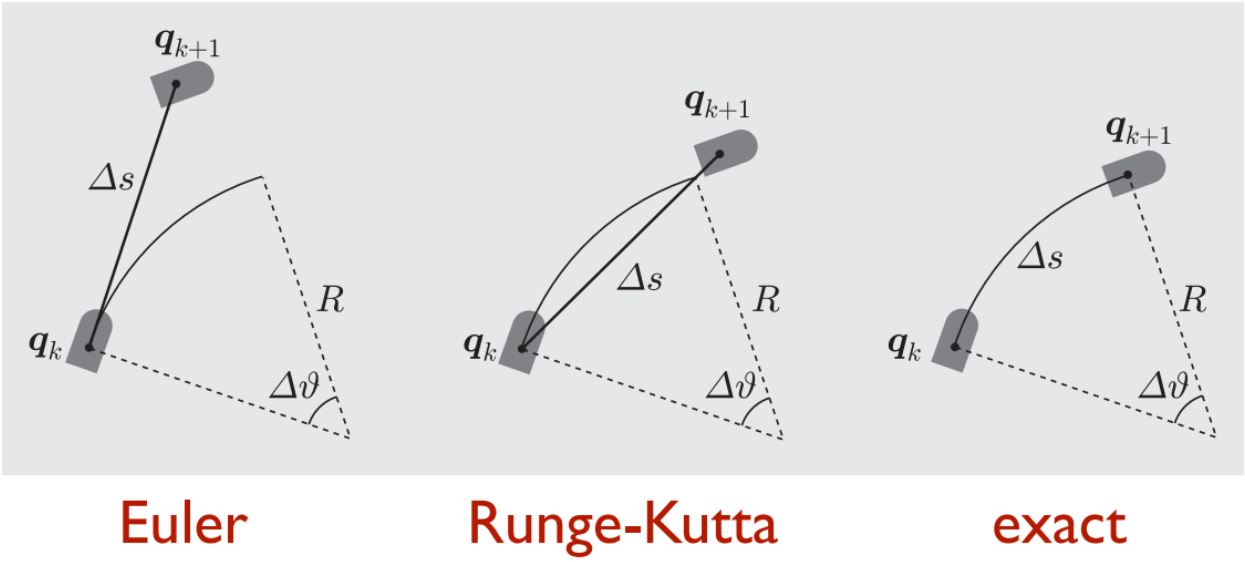


Figure 1: Odometric localization of a unicycle over one arc of circle (redrawn). Euler moves along the initial heading; Runge–Kutta moves along the *mean* heading; the exact formula follows the arc. The small grey circle marks the exact end point. The arc here is exaggerated ($\Delta\theta = 60°$); at realistic sampling rates the three points almost coincide.

Proposition 2.4 (Leading-order error of the schemes)**.** *Let $a = \omega_k T_s/2$. The exact update equals the Runge–Kutta one with the step length scaled by* $\operatorname{sinc}(a) = \sin a/a$*:*

$$\begin{aligned} x_{k+1} - x_k &= v_k T_s \operatorname{sinc}(a) \cos(\theta_k + a), \\ y_{k+1} - y_k &= v_k T_s \operatorname{sinc}(a) \sin(\theta_k + a). \end{aligned} \tag{5}$$

Consequently, to leading order the position error per step *is $\frac{1}{2} v_k \omega_k T_s^2$ for Euler and $\frac{1}{24} v_k \omega_k^2 T_s^3$ for Runge–Kutta.*

Proof. Use $\sin\alpha - \sin\beta = 2\cos\frac{\alpha+\beta}{2}\sin\frac{\alpha-\beta}{2}$ with $\alpha = \theta_{k+1}$, $\beta = \theta_k$: $\frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k) = \frac{2v_k}{\omega_k}\sin(a)\cos(\theta_k + a) = v_k T_s \operatorname{sinc}(a)\cos(\theta_k + a)$. The y formula is analogous with $\cos\beta - \cos\alpha = 2\sin\frac{\alpha+\beta}{2}\sin\frac{\alpha-\beta}{2}$. Now $\operatorname{sinc}(a) = 1 - a^2/6 + \cdots = 1 - (\omega_k T_s)^2/24 + \ldots$, so Runge–Kutta (which sets sinc $= 1$) errs by about $v_k T_s\,(\omega_k T_s)^2/24$. Euler uses the heading θ_k instead of $\theta_k + a$: its step vector is rotated by the angle a with respect to the exact one, giving an error of about $v_k T_s\, a = \frac{1}{2} v_k \omega_k T_s^2$. □

Intuition

Euler commits to the *starting* heading, Runge–Kutta to the *average* heading, and the exact formula additionally knows that a chord is shorter than its arc (the sinc factor). Taking the heading at the middle of the interval is what removes the dominant error: halve T_s and the Euler error per step drops by a factor 4, the Runge–Kutta one by a factor 8 (Fig. 2). This is why the differences "tend to disappear as the sampling rate increases".

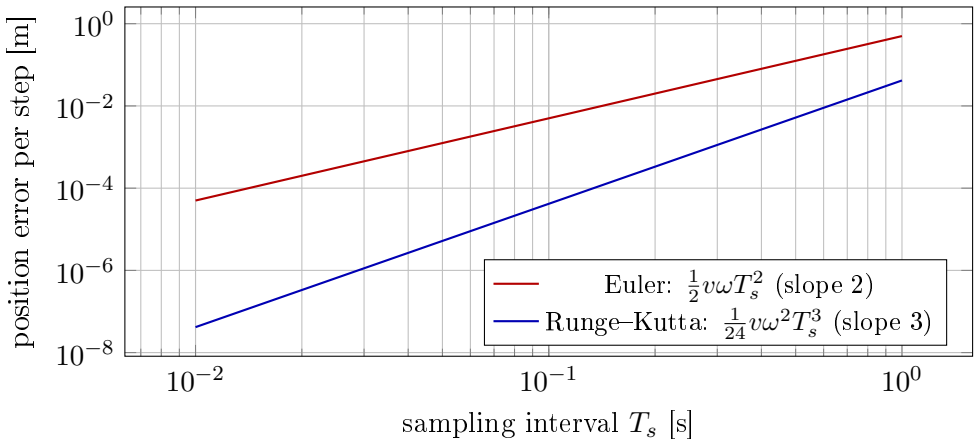


Figure 2: Leading-order local error of the two approximate schemes for $v = 1$ m/s, $\omega = 1$ rad/s (illustrative asymptotics from Prop. 2.4).

Example 2.5 (A coarse step, numerically)**.** Take $\theta_k = 0$, $v_k = 1$ m/s, $\omega_k = \pi/2$ rad/s and $T_s = 1$ s (a quarter turn in one step, deliberately coarse).

Scheme	$(x_{k+1} - x_k,\ y_{k+1} - y_k)$ [m]	distance from exact
Euler	$(1,\ 0)$	0.733
Runge–Kutta	$(0.707,\ 0.707)$	0.100
Exact	$(2/\pi,\ 2/\pi) = (0.637,\ 0.637)$	0

Check with (5): $a = \pi/4$, $\mathrm{sinc}(a) = 0.900$, and $0.900 \cdot 0.707 = 0.637$.

3 Reconstructing the inputs from proprioceptive sensors *(odometry deck, slide 7)*

The formulas above assume that v_k, ω_k are known. But no actuation system is ideal: the *commanded* inputs are not what the robot actually executes. It is therefore better to reconstruct v_k and ω_k from their *effect*:

$$v_k T_s = \Delta s, \qquad \omega_k T_s = \Delta\theta, \qquad \frac{v_k}{\omega_k} = \frac{\Delta s}{\Delta\theta}, \tag{6}$$

where Δs is the signed distance traveled and $\Delta\theta$ the total orientation change over the interval, both measured by *proprioceptive* sensors.

Proposition 3.1 (Differential drive)**.** *For a differential-drive robot with wheel radius r and wheel separation d, let $\Delta\phi_R, \Delta\phi_L$ be the total rotations of the right and left wheel measured by the encoders during the sampling interval. Then*

$$\Delta s = \frac{r}{2}(\Delta\phi_R + \Delta\phi_L), \qquad \Delta\theta = \frac{r}{d}(\Delta\phi_R - \Delta\phi_L). \tag{7}$$

Proof. The wheel rim speeds are $r\dot{\phi}_R$ and $r\dot{\phi}_L$. The robot center moves with the average rim speed, $v = \frac{r}{2}(\dot{\phi}_R + \dot{\phi}_L)$, and the robot turns at the rim-speed difference divided by the separation, $\omega = \frac{r}{d}(\dot{\phi}_R - \dot{\phi}_L)$. Integrating over the sampling interval gives (7). □

With (6)–(7) the three schemes can be written without T_s:

$$\text{Euler: } x_{k+1} = x_k + \Delta s\cos\theta_k, \qquad \text{RK: } x_{k+1} = x_k + \Delta s\cos\!\left(\theta_k + \tfrac{\Delta\theta}{2}\right),$$
$$\text{exact: } x_{k+1} = x_k + \tfrac{\Delta s}{\Delta\theta}(\sin\theta_{k+1} - \sin\theta_k),$$

and analogously for y; $\theta_{k+1} = \theta_k + \Delta\theta$. Similar formulas can be derived for other WMRs, e.g. the bicycle.

Algorithm 3.2 (One odometry update)**.** Given $\boldsymbol{q}_k$ and the encoder increments $\Delta\phi_R, \Delta\phi_L$:

1. compute Δs and $\Delta\theta$ from (7);
2. set $\theta_{k+1} = \theta_k + \Delta\theta$;
3. if $|\Delta\theta| < \varepsilon$ (conditional instruction) use the straight-line/Runge–Kutta update for x_{k+1}, y_{k+1}; otherwise use the exact formulas (4) with $v_k/\omega_k = \Delta s/\Delta\theta$.

Example 3.3. With $r = 0.05$ m, $d = 0.30$ m, $\Delta\phi_R = 0.50$ rad, $\Delta\phi_L = 0.30$ rad: $\Delta s = \frac{0.05}{2}(0.80) = 0.020$ m and $\Delta\theta = \frac{0.05}{0.30}(0.20) = 0.0333$ rad $\approx 1.9°$. The robot advanced 2 cm while turning slightly left, because the right wheel rolled more than the left.

Intuition

The encoders "see" only the wheels. The sum of the two wheel rotations says how far the midpoint of the axle moved; their *difference* says how much the vehicle swung around, and it is divided by the small separation d — a lever arm effect that makes heading very sensitive to wheel errors (we will see the consequence in the next section).

4 Dead reckoning and odometric drift *(odometry deck, slides 8–9)*

Definition 4.1 (Odometric localization / dead reckoning)**.** Maintaining an estimate of the robot configuration by *iterative integration of the kinematic model*, with velocities reconstructed from wheel encoders, starting from an estimate of $\boldsymbol{q}_0$.

Its accuracy cannot be better than that of $\boldsymbol{q}_0$. Moreover, odometric errors *accumulate over time* (*odometric drift*), so accuracy deteriorates along sufficiently long paths. The causes are:

- the *integration error* of Euler/Runge–Kutta (reducible, Sect. 2);
- *wheel slippage* and irregular ground (a model perturbation);
- *inaccurate calibration* of kinematic parameters such as the wheel radius (a model uncertainty);
- *encoder noise* (a measurement uncertainty).

The last three are inevitable, and the achievable performance also depends on the kinematic arrangement of the robot (e.g. differential drive is usually better than synchro drive). Effective localization therefore uses exteroceptive sensing as well as proprioceptive sensing (Part II).

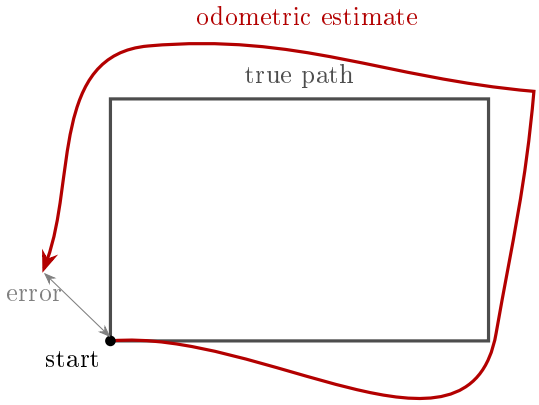


Figure 3: Conceptual sketch of odometric drift (original): the robot drives around a rectangle and returns to its start, but the integrated estimate does not close the loop; the gap keeps growing with distance.

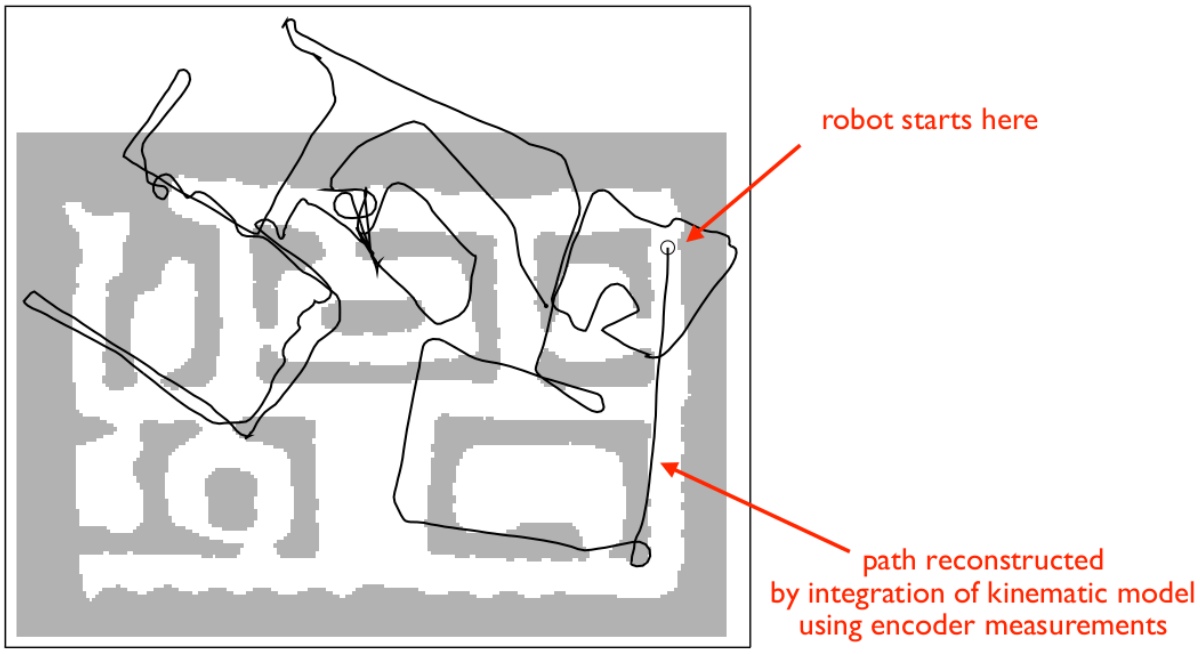


Figure 4: Real experiment: path reconstructed by integrating the kinematic model with encoder measurements, drawn over the environment map

Example 4.2 (Why a 1% calibration error is not small)**.** Differential drive with $d = 0.30$ m. Suppose the true right wheel is 1% larger than the nominal radius used by the odometry, and the robot is commanded to go straight, so both encoders report the same rotation $\Delta\phi$. The odometry concludes $\Delta\theta = 0$, but physically $\Delta\theta = \frac{0.01\, r\Delta\phi}{d} \approx \frac{0.01}{0.30}\Delta s \approx 0.033\,\Delta s$: the robot actually follows an arc with curvature ≈ 0.033 m^{-1} (radius ≈ 30 m). After $s = 10$ m the heading is off by ≈ 0.33 rad ($\approx 19°$) and the lateral position by $\approx s^2\kappa/2 \approx 1.7$ m, while the estimate still believes the robot is on the straight line.

Intuition

A tiny mismatch between the wheels is divided by the small wheelbase d and becomes a heading error; a heading error then is turned into a sideways position error that grows with the distance traveled. Errors are never "forgotten" by integration — they only add up. To stop this we need a measurement that does not depend on the past: a landmark, a wall, a map.

5 From dead reckoning to probabilistic localization *(EKF deck, slide 2)*

Dead reckoning keeps a *single hypothesis* about the configuration, and its error diverges over time. *Probabilistic localization* instead maintains a probability distribution over the space of all possible hypotheses. If a map of the environment is known, the actual readings of an exteroceptive sensor can be compared with the readings predicted from the current estimate; the mismatch is used to reduce the odometric drift. This idea is formalized by Bayesian estimation, and a tractable instance is the *Extended Kalman Filter* (EKF).

Intuition

Walking in a dark corridor you count steps (odometry) and your belief about your position spreads out. When your hand touches a door frame you know exists on the map, the belief suddenly shrinks. The filter is the bookkeeping that tells you exactly how much to trust the step count versus the touch.

6 Basic concepts: mean, covariance, Gaussians *(EKF deck, slides 3–4)*

Definition 6.1 (Mean, covariance, multivariate Gaussian)**.** Let $\boldsymbol{X} \in \mathbb{R}^n$ be a random vector with density $f_{\boldsymbol{X}}(\boldsymbol{x})$. Its *expected value* and *covariance matrix* are

$$\bar{\boldsymbol{X}} = \mathbb{E}(\boldsymbol{X}) = \int_{\mathbb{R}^n} \boldsymbol{x}\, f_{\boldsymbol{X}}(\boldsymbol{x})\, d\boldsymbol{x}, \qquad \mathbf{P}_X = \mathbb{E}\big((\boldsymbol{X} - \bar{\boldsymbol{X}})(\boldsymbol{X} - \bar{\boldsymbol{X}})^\top\big).$$

$\boldsymbol{X}$ is *multivariate Gaussian* if

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^n |\mathbf{P}_X|}}\, \exp\Big(-\tfrac{1}{2}(\boldsymbol{x} - \bar{\boldsymbol{X}})^\top \mathbf{P}_X^{-1} (\boldsymbol{x} - \bar{\boldsymbol{X}})\Big).$$

Proposition 6.2 (Geometry of a Gaussian)**.** *The equidensity contours of a Gaussian are ellipsoids* $(\boldsymbol{x} - \bar{\boldsymbol{X}})^\top \mathbf{P}_X^{-1} (\boldsymbol{x} - \bar{\boldsymbol{X}}) = c$*. Their principal axes are directed along the eigenvectors of* $\mathbf{P}_X$*, and the squared lengths of the axes are proportional to the corresponding eigenvalues.*

Proof. Write $\mathbf{P}_X = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ with $\mathbf{U}$ orthogonal. In the coordinates $\boldsymbol{z} = \mathbf{U}^\top(\boldsymbol{x} - \bar{\boldsymbol{X}})$ the contour is $\sum_i z_i^2/\lambda_i = c$, an axis-aligned ellipsoid with semi-axes $\sqrt{c\lambda_i}$ along the columns of $\mathbf{U}$. □

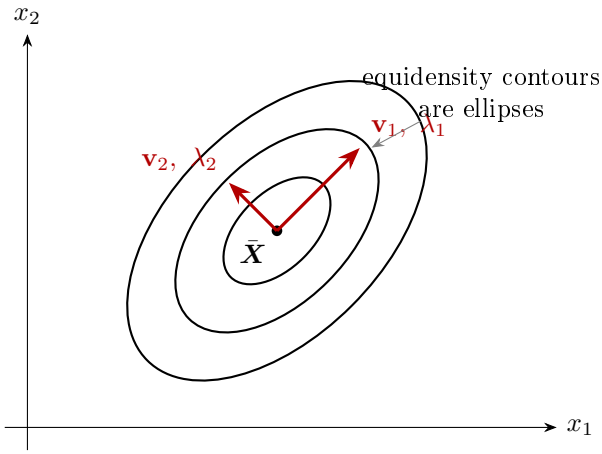


Figure 5: Geometric interpretation of a covariance matrix (redrawn): ellipses centered at the mean, with axes along the eigenvectors $\mathbf{v}_i$ of $\mathbf{P}_X$. Drawn for $\mathbf{P}_X = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, $\lambda_1 = 3$, $\lambda_2 = 1$.

Intuition

The covariance matrix is the "shape of the cloud": a long thin ellipse means "I am quite sure in one direction and quite unsure in the other". Example: $\mathbf{P}_X = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ has eigenvalues 3 and 1 with eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$, so the cloud is stretched along the diagonal, and its length along that direction is $\sqrt{3} \approx 1.7$ times the length across it. The off-diagonal term $1 > 0$ says that x_1 and x_2 tend to be large or small together. Filters will propagate exactly this kind of ellipse.

7 Kalman filter without noise *(EKF deck, slides 5–9)*

7.1 Recursive observer: prediction and correction

Consider the linear discrete-time system *without noise*

$$\boldsymbol{x}_{k+1} = \mathbf{A}_k \boldsymbol{x}_k + \mathbf{B}_k \boldsymbol{u}_k, \qquad \boldsymbol{y}_k = \mathbf{C}_k \boldsymbol{x}_k. \tag{8}$$

We want a *recursive observer*: an estimate $\hat{\boldsymbol{x}}_{k+1}$ of $\boldsymbol{x}_{k+1}$ computed from $\boldsymbol{u}_k$, $\boldsymbol{y}_{k+1}$ and the previous estimate $\hat{\boldsymbol{x}}_k$, in two steps:

1. **prediction**: build an intermediate estimate $\hat{\boldsymbol{x}}_{k+1|k}$ by propagating $\hat{\boldsymbol{x}}_k$ through the process dynamics,
$$\hat{\boldsymbol{x}}_{k+1|k} = \mathbf{A}_k \hat{\boldsymbol{x}}_k + \mathbf{B}_k \boldsymbol{u}_k; \tag{9}$$
2. **correction (update)**: correct the prediction using the difference between the measured and the predicted output.

Even without noise a correction is needed, because the exact initial state $\boldsymbol{x}_0$ is unknown.

7.2 The correction as a constrained optimization

To be consistent with the measured output, $\boldsymbol{x}_{k+1}$ must belong to the hyperplane

$$\Omega = \{\boldsymbol{x} : \ \mathbf{C}_{k+1} \boldsymbol{x} = \boldsymbol{y}_{k+1}\}, \tag{10}$$

so the correction $\Delta\boldsymbol{x}$ must satisfy $\mathbf{C}_{k+1}(\hat{\boldsymbol{x}}_{k+1|k} + \Delta\boldsymbol{x}) = \boldsymbol{y}_{k+1}$. There are infinitely many such corrections; intuitively the best is the *closest* to the prediction, which we believe is accurate (Fig. 6).

Proposition 7.1 (Minimum-norm correction). *Assume $\mathbf{C}_{k+1}$ has full row rank. The solution*

of

$$\min \|\Delta \boldsymbol{x}\| \quad s.t. \quad \mathbf{C}_{k+1}\Delta \boldsymbol{x} = \boldsymbol{y}_{k+1} - \mathbf{C}_{k+1}\hat{\boldsymbol{x}}_{k+1|k}$$

is

$$\Delta \boldsymbol{x} = \mathbf{C}_{k+1}^{\dagger}\,\boldsymbol{\nu}_{k+1}, \qquad \mathbf{C}_{k+1}^{\dagger} = \mathbf{C}_{k+1}^{\top}(\mathbf{C}_{k+1}\mathbf{C}_{k+1}^{\top})^{-1}, \qquad \boldsymbol{\nu}_{k+1} = \boldsymbol{y}_{k+1} - \mathbf{C}_{k+1}\hat{\boldsymbol{x}}_{k+1|k}, \tag{11}$$

where $\mathbf{C}^{\dagger}$ *is the* pseudoinverse *and* $\boldsymbol{\nu}_{k+1}$ *is the* innovation.

Proof. Lagrangian $\frac{1}{2}\Delta \boldsymbol{x}^{\top}\Delta \boldsymbol{x} + \boldsymbol{\lambda}^{\top}(\boldsymbol{\nu} - \mathbf{C}\Delta \boldsymbol{x})$. Stationarity gives $\Delta \boldsymbol{x} = \mathbf{C}^{\top}\boldsymbol{\lambda}$; substituting in the constraint, $\boldsymbol{\nu} = \mathbf{C}\mathbf{C}^{\top}\boldsymbol{\lambda}$, so $\boldsymbol{\lambda} = (\mathbf{C}\mathbf{C}^{\top})^{-1}\boldsymbol{\nu}$ (invertible by full row rank) and $\Delta \boldsymbol{x} = \mathbf{C}^{\top}(\mathbf{C}\mathbf{C}^{\top})^{-1}\boldsymbol{\nu}$. □

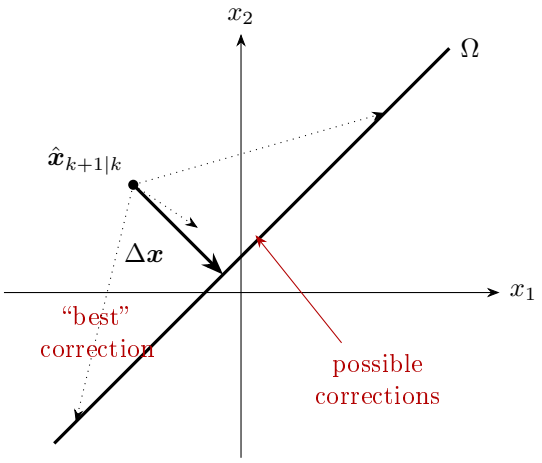


Figure 6: Naive correction (redrawn): among all points of the measurement hyperplane Ω, pick the one closest to the prediction; $\Delta \boldsymbol{x}$ is orthogonal to Ω.

Intuition

The measurement tells you "the true state is somewhere on this line". You were already standing at the prediction, so you take the shortest step onto the line — a perpendicular projection. *Worked numbers:* $\boldsymbol{x} \in \mathbb{R}^2$, $\mathbf{C} = (1 \;\; 1)$ (we only measure the sum $x_1 + x_2$), $\hat{\boldsymbol{x}}_{k+1|k} = (1, 0)$ and $y_{k+1} = 3$. Then $\nu = 3 - 1 = 2$, $\mathbf{C}^{\dagger} = (\frac{1}{2}, \frac{1}{2})^{\top}$, $\Delta \boldsymbol{x} = (1, 1)$ and $\hat{\boldsymbol{x}}_{k+1} = (2, 1)$, whose sum is indeed 3.

7.3 Why this naive observer is not enough

The resulting two-step observer is $\hat{\boldsymbol{x}}_{k+1|k} = \mathbf{A}_k\hat{\boldsymbol{x}}_k + \mathbf{B}_k\boldsymbol{u}_k$, $\hat{\boldsymbol{x}}_{k+1} = \hat{\boldsymbol{x}}_{k+1|k} + \mathbf{C}_{k+1}^{\dagger}\boldsymbol{\nu}_{k+1}$. In general $\hat{\boldsymbol{x}}_{k+1}$ will *not* converge to $\boldsymbol{x}_{k+1}$: the correction is naive because estimation errors directed as Ω are not corrected. In particular the gain does not exploit the observability of the system (unlike, e.g., a Luenberger observer).

Intuition

Take $\mathbf{A}_k = \mathbf{I}$, $\mathbf{C} = (1 \;\; 1)$, true state $(1, 2)$ and $\hat{\boldsymbol{x}}_0 = (0, 0)$. The first correction gives $(1.5, 1.5)$, which already reproduces $y = 3$, so every later innovation is zero and the error $(-0.5, 0.5)$ never disappears: a correction that only looks at the "compatible with the measurement" direction cannot fix errors along Ω. (This pair is unobservable, so no observer could do better; the point is only to see the mechanism.) The remedy is to choose the gain in a principled way using noise statistics.

8 Kalman filter with process noise only *(EKF deck, slides 10–15)*

Now include *process noise*:

$$\boldsymbol{x}_{k+1} = \mathbf{A}_k \boldsymbol{x}_k + \mathbf{B}_k \boldsymbol{u}_k + \boldsymbol{v}_k, \qquad \boldsymbol{y}_k = \mathbf{C}_k \boldsymbol{x}_k, \tag{12}$$

where $\boldsymbol{v}_k$ is *white* (successive samples are uncorrelated), *Gaussian*, zero-mean with covariance $\mathbf{V}_k$. The system is now a random process, so we compute both an estimate $\hat{\boldsymbol{x}}_{k+1}$ and its estimation-error covariance $\mathbf{P}_{k+1}$, keeping the prediction/correction structure.

8.1 Prediction

The state prediction is unchanged, $\hat{\boldsymbol{x}}_{k+1|k} = \mathbf{A}_k \hat{\boldsymbol{x}}_k + \mathbf{B}_k \boldsymbol{u}_k$, because $\boldsymbol{v}_k$ has zero mean.

Lemma 8.1 (Covariance prediction)**.** $\mathbf{P}_{k+1|k} = \mathbf{A}_k \mathbf{P}_k \mathbf{A}_k^\top + \mathbf{V}_k$.

Proof. $\boldsymbol{x}_{k+1} - \hat{\boldsymbol{x}}_{k+1|k} = \mathbf{A}_k(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k) + \boldsymbol{v}_k$. Expanding $\mathbf{P}_{k+1|k} = \mathbb{E}\big((\boldsymbol{x}_{k+1} - \hat{\boldsymbol{x}}_{k+1|k})(\cdot)^\top\big)$ gives $\mathbf{A}_k \mathbb{E}\big((\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k)(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k)^\top\big)\mathbf{A}_k^\top + \mathbb{E}(\boldsymbol{v}_k \boldsymbol{v}_k^\top)$ plus cross terms. By linearity of $\mathbb{E}$ and the independence of $\boldsymbol{v}_k$ from $\hat{\boldsymbol{x}}_k$ and $\boldsymbol{x}_k$ (and zero mean), the cross terms vanish. Hence $\mathbf{P}_{k+1|k} = \mathbf{A}_k \mathbf{P}_k \mathbf{A}_k^\top + \mathbf{V}_k$. □

The predicted state distribution is Gaussian with mean $\hat{\boldsymbol{x}}_{k+1|k}$ and covariance $\mathbf{P}_{k+1|k}$.

Intuition

Propagating through $\mathbf{A}_k$ stretches or rotates the uncertainty ellipse; adding $\mathbf{V}_k$ *fattens* it: every step, the world shakes the robot a little and we lose a bit of knowledge. With no measurements, $\mathbf{P}$ can only grow — the covariance-level picture of odometric drift.

8.2 State correction: Mahalanobis distance and weighted pseudoinverse

We should choose $\Delta\boldsymbol{x}$ so as to get the *most likely* $\boldsymbol{x}$ in Ω, i.e. the $\boldsymbol{x}$ that maximizes the Gaussian $p(\boldsymbol{x}) \propto \exp\big(-\frac{1}{2}(\boldsymbol{x} - \hat{\boldsymbol{x}}_{k+1|k})^\top \mathbf{P}_{k+1|k}^{-1}(\boldsymbol{x} - \hat{\boldsymbol{x}}_{k+1|k})\big)$. Since p is maximized when the exponent is minimized, the correction minimizes the squared *Mahalanobis distance*

$$\|\Delta\boldsymbol{x}\|_M^2 = \Delta\boldsymbol{x}^\top \mathbf{P}_{k+1|k}^{-1} \Delta\boldsymbol{x}, \tag{13}$$

which penalizes deviations along directions where the precision is higher.

Proposition 8.2 (Weighted correction)**.** *The solution of* $\min \|\Delta\boldsymbol{x}\|_M$ *s.t.* $\mathbf{C}_{k+1}\Delta\boldsymbol{x} = \boldsymbol{y}_{k+1} - \mathbf{C}_{k+1}\hat{\boldsymbol{x}}_{k+1|k}$ *is*

$$\Delta\boldsymbol{x} = \mathbf{C}_{k+1,M}^\dagger \boldsymbol{\nu}_{k+1}, \qquad \mathbf{C}_{k+1,M}^\dagger = \mathbf{P}_{k+1|k}\mathbf{C}_{k+1}^\top \big(\mathbf{C}_{k+1}\mathbf{P}_{k+1|k}\mathbf{C}_{k+1}^\top\big)^{-1}, \tag{14}$$

the weighted pseudoinverse *of* $\mathbf{C}_{k+1}$.

Proof. As in Prop. 7.1 with the Lagrangian $\frac{1}{2}\Delta\boldsymbol{x}^\top\mathbf{P}^{-1}\Delta\boldsymbol{x} + \boldsymbol{\lambda}^\top(\boldsymbol{\nu} - \mathbf{C}\Delta\boldsymbol{x})$: stationarity gives $\Delta\boldsymbol{x} = \mathbf{P}\mathbf{C}^\top\boldsymbol{\lambda}$, the constraint gives $\boldsymbol{\nu} = \mathbf{C}\mathbf{P}\mathbf{C}^\top\boldsymbol{\lambda}$, hence $\Delta\boldsymbol{x} = \mathbf{P}\mathbf{C}^\top(\mathbf{C}\mathbf{P}\mathbf{C}^\top)^{-1}\boldsymbol{\nu}$. □

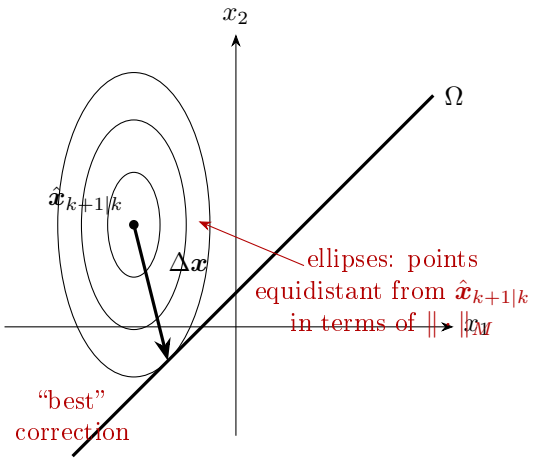


Figure 7: Weighted correction (redrawn): the best correction is the point of Ω reached by the smallest ellipse of the current covariance estimate, i.e. where an ellipse is tangent to Ω. Here $\mathbf{P}_{k+1|k} = \mathrm{diag}(0.55^2, 1.1^2)$, $\mathbf{C} = (-1 \quad 1)$.

Intuition

Now "close" is measured with the ruler of the filter's own uncertainty: a unit of error along a well-known direction costs a lot, along a poorly known direction it is cheap. So the filter moves mostly along the direction it is *least sure* about. *Worked numbers:* same $\mathbf{C} = (1 \quad 1)$, $\nu = 2$ as before, but $\mathbf{P}_{k+1|k} = \mathrm{diag}(4, 1)$ (x_1 is more uncertain). Then $\mathbf{CPC}^\top = 5$ and $\Delta\boldsymbol{x} = \mathbf{PC}^\top\nu/5 = (1.6,\, 0.4)$ instead of the isotropic $(1, 1)$: the uncertain coordinate absorbs 80% of the correction.

8.3 Covariance correction and its flaw

Proposition 8.3 (Covariance correction)**.** *With the correction* (14), $\mathbf{P}_{k+1} = \mathbf{P}_{k+1|k} - \mathbf{C}_{k+1,M}^\dagger \mathbf{C}_{k+1}\mathbf{P}_{k+1|k}$.

Proof. Since $\boldsymbol{y}_{k+1} = \mathbf{C}_{k+1}\boldsymbol{x}_{k+1}$, $\boldsymbol{\nu}_{k+1} = \mathbf{C}_{k+1}(\boldsymbol{x}_{k+1} - \hat{\boldsymbol{x}}_{k+1|k})$ and the error after the correction is $(\mathbf{I} - \mathbf{C}_M^\dagger\mathbf{C})(\boldsymbol{x}_{k+1} - \hat{\boldsymbol{x}}_{k+1|k})$ (indices dropped). With $\mathbf{S} = \mathbf{CPC}^\top$, $\mathbf{C}_M^\dagger = \mathbf{PC}^\top\mathbf{S}^{-1}$ and $\mathbf{X} := \mathbf{PC}^\top\mathbf{S}^{-1}\mathbf{CP}$ (symmetric), we get $\mathbf{P}_{k+1} = (\mathbf{I} - \mathbf{C}_M^\dagger\mathbf{C})\mathbf{P}(\mathbf{I} - \mathbf{C}_M^\dagger\mathbf{C})^\top = \mathbf{P} - \mathbf{X} - \mathbf{X} + \mathbf{PC}^\top\mathbf{S}^{-1}\mathbf{SS}^{-1}\mathbf{CP} = \mathbf{P} - \mathbf{X} = \mathbf{P} - \mathbf{C}_M^\dagger\mathbf{CP}$. □

The resulting two-step filter is: prediction (9) and $\mathbf{P}_{k+1|k} = \mathbf{A}_k\mathbf{P}_k\mathbf{A}_k^\top + \mathbf{V}_k$; correction $\hat{\boldsymbol{x}}_{k+1} = \hat{\boldsymbol{x}}_{k+1|k} + \mathbf{C}_{k+1,M}^\dagger\boldsymbol{\nu}_{k+1}$ and $\mathbf{P}_{k+1} = \mathbf{P}_{k+1|k} - \mathbf{C}_{k+1,M}^\dagger\mathbf{C}_{k+1}\mathbf{P}_{k+1|k}$.

Problem: with no measurement noise the covariance estimate becomes *singular*: $\mathbf{CP}_{k+1} = \mathbf{CP} - \mathbf{CPC}^\top\mathbf{S}^{-1}\mathbf{CP} = \mathbf{0}$, so there is no uncertainty left in the direction normal to the measurement hyperplane.

Intuition

Continue the example $\mathbf{P}_{k+1|k} = \mathrm{diag}(4, 1)$, $\mathbf{C} = (1 \quad 1)$: $\mathbf{C}_M^\dagger\mathbf{CP} = \frac{1}{5}\left(\begin{smallmatrix}16 & 4\\ 4 & 1\end{smallmatrix}\right)$, so $\mathbf{P}_{k+1} = \left(\begin{smallmatrix}0.8 & -0.8\\ -0.8 & 0.8\end{smallmatrix}\right)$, with determinant 0. The filter claims to know $x_1 + x_2$ *exactly* (variance 0 along $(1, 1)$) — reasonable if the sensor is perfect, but a real sensor never is. That is why we now add measurement noise.

9 Kalman filter: the full version *(EKF deck, slides 16–21)*

Finally include also *measurement (sensor) noise*:

$$\boldsymbol{x}_{k+1} = \mathbf{A}_k\boldsymbol{x}_k + \mathbf{B}_k\boldsymbol{u}_k + \boldsymbol{v}_k, \qquad \boldsymbol{y}_k = \mathbf{C}_k\boldsymbol{x}_k + \boldsymbol{w}_k, \tag{15}$$

with $\boldsymbol{v}_k, \boldsymbol{w}_k$ white Gaussian, zero mean, covariances $\mathbf{V}_k, \mathbf{W}_k$. The dynamic equation is unchanged, so the predictions are the same: $\hat{\boldsymbol{x}}_{k+1|k} = \mathbf{A}_k\hat{\boldsymbol{x}}_k + \mathbf{B}_k\boldsymbol{u}_k$ and $\mathbf{P}_{k+1|k} = \mathbf{A}_k\mathbf{P}_k\mathbf{A}_k^\top + \mathbf{V}_k$.

9.1 State correction with a noisy measurement

Because of the sensor noise the output is no longer certain: we only know that $\boldsymbol{y}_{k+1}$ is drawn from a Gaussian with mean $\mathbf{C}_{k+1}\boldsymbol{x}_{k+1}$ and covariance $\mathbf{W}_{k+1}$. The correction therefore proceeds as follows.

1. Compute the *most likely output value* $\boldsymbol{y}^*_{k+1}$, given the predictions and the measured output $\boldsymbol{y}_{k+1}$.
2. Compute the associated *most likely hyperplane* $\Omega^* = \{\boldsymbol{x} : \ \mathbf{C}_{k+1}\boldsymbol{x} = \boldsymbol{y}^*_{k+1}\}$, a compromise between prediction and measurement.
3. Compute $\Delta\boldsymbol{x}$ as before, but using Ω^* in place of Ω.

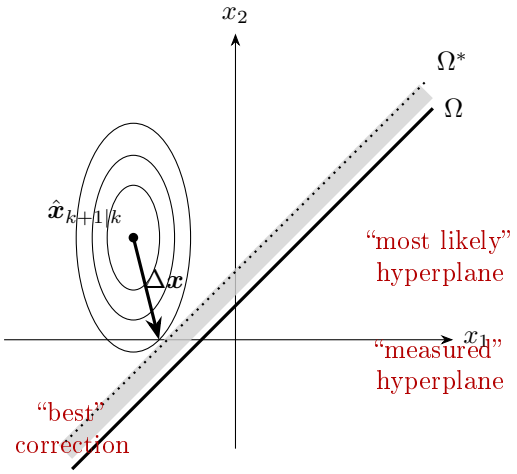


Figure 8: Correction with sensor noise (redrawn): the measured hyperplane Ω is uncertain (grey band); the correction targets the most likely hyperplane Ω^*, which lies between the prediction and Ω. Illustrative numbers for $\mathbf{W} = 0.5$.

Theorem 9.1 (Kalman filter)**.** *For the system* (15) *the Kalman filter (KF) is*

$$\begin{aligned}
\hat{\boldsymbol{x}}_{k+1|k} &= \mathbf{A}_k\hat{\boldsymbol{x}}_k + \mathbf{B}_k\boldsymbol{u}_k, \\
\mathbf{P}_{k+1|k} &= \mathbf{A}_k\mathbf{P}_k\mathbf{A}_k^\top + \mathbf{V}_k, \\
\hat{\boldsymbol{x}}_{k+1} &= \hat{\boldsymbol{x}}_{k+1|k} + \mathbf{R}_{k+1}\boldsymbol{\nu}_{k+1}, \qquad \boldsymbol{\nu}_{k+1} = \boldsymbol{y}_{k+1} - \mathbf{C}_{k+1}\hat{\boldsymbol{x}}_{k+1|k}, \\
\mathbf{P}_{k+1} &= \mathbf{P}_{k+1|k} - \mathbf{R}_{k+1}\mathbf{C}_{k+1}\mathbf{P}_{k+1|k},
\end{aligned} \tag{16}$$

with the Kalman gain matrix

$$\mathbf{R}_{k+1} = \mathbf{P}_{k+1|k}\mathbf{C}_{k+1}^\top\big(\mathbf{C}_{k+1}\mathbf{P}_{k+1|k}\mathbf{C}_{k+1}^\top + \mathbf{W}_{k+1}\big)^{-1}. \tag{17}$$

Sketch of the gain. Let $\mathbf{S} = \mathbf{C}\mathbf{P}_{k+1|k}\mathbf{C}^\top$ be the covariance of the predicted output. Fusing the predicted output $\mathcal{N}(\mathbf{C}\hat{\boldsymbol{x}}_{k+1|k}, \mathbf{S})$ with the measurement (noise $\mathbf{W}$) gives the most likely output $\boldsymbol{y}^* = \mathbf{C}\hat{\boldsymbol{x}}_{k+1|k} + \mathbf{S}(\mathbf{S}+\mathbf{W})^{-1}\boldsymbol{\nu}$. Applying Prop. 8.2 with Ω^*: $\Delta\boldsymbol{x} = \mathbf{P}\mathbf{C}^\top\mathbf{S}^{-1}(\boldsymbol{y}^* - \mathbf{C}\hat{\boldsymbol{x}}_{k+1|k}) = \mathbf{P}\mathbf{C}^\top\mathbf{S}^{-1}\mathbf{S}(\mathbf{S}+\mathbf{W})^{-1}\boldsymbol{\nu} = \mathbf{R}_{k+1}\boldsymbol{\nu}$. If $\mathbf{W} \to 0$, $\mathbf{R}_{k+1} \to \mathbf{C}^\dagger_{k+1,M}$ and we recover the previous filter. □

Intuition

$\mathbf{R}$ decides *how much of the surprise* $\boldsymbol{\nu}$ *to believe*. If $\mathbf{R}$ is "large" the measurements are more reliable than the prediction; if $\mathbf{R}$ is "small" the prediction is more reliable. Scalar example: x with $c = 1$, prior variance $P_{k+1|k} = 4$, sensor noise $W = 1$. Then $R = \frac{4}{4+1} = 0.8$. If the prediction is 0 and the sensor reads 5, the update is $\hat{x}_{k+1} = 0 + 0.8 \cdot 5 = 4$ (mostly the sensor), and $P_{k+1} = 4 - 0.8 \cdot 4 = 0.8$, smaller than both 4 and 1: fusing two uncertain pieces of information always yields something more certain than either. With a very noisy sensor ($W = 100$) we would get $R \approx 0.04$ and the prediction would dominate.

9.2 Properties of the KF

- relatively low computational load, plus a lot of quantities can be precomputed; no measurements need to be stored;
- it is a *minimum mean-square error* (MMSE) estimator: it minimizes $\|\mathbb{E}(\boldsymbol{x}_{k+1} - \hat{\boldsymbol{x}}_{k+1})\|^2$ for all k;
- it is *correct*: it provides mean value and covariance of the *posterior* Gaussian distribution;
- if the noises are non-Gaussian, the KF is still the best *linear* estimator, but more accurate nonlinear filters may exist;
- if the system is time-invariant and detectable, the mean-square estimation error stays bounded; it generally does not converge to zero in the presence of persistent process noise.

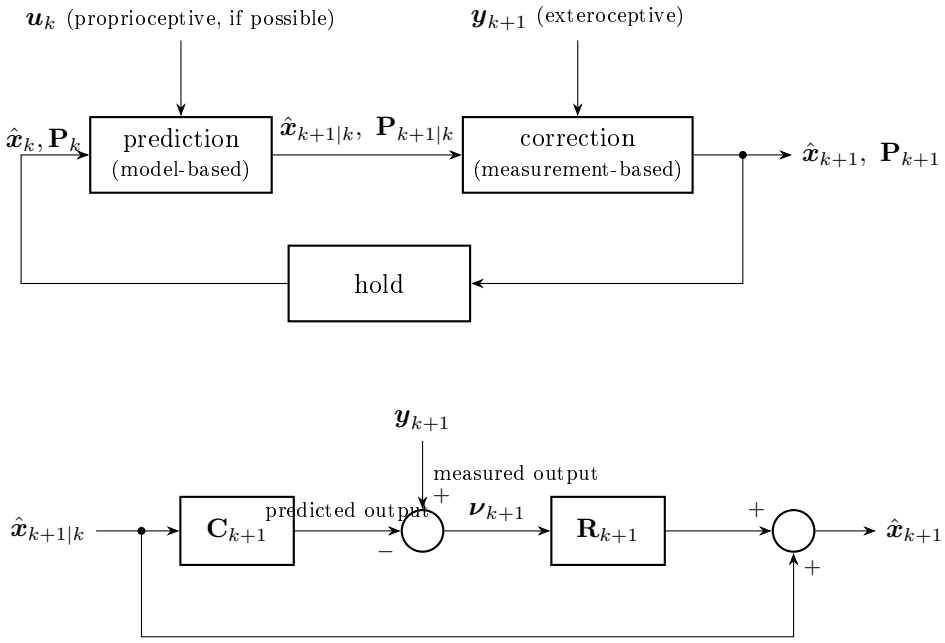


Figure 9: Wrapping up (redrawn). Top: the filter alternates a model-based prediction, driven by proprioceptive data when available, and a measurement-based correction driven by exteroceptive data; the output is held for one step and fed back. Bottom: the correction block: the innovation $\boldsymbol{\nu}_{k+1}$ (measured minus predicted output) is weighed by the gain $\mathbf{R}_{k+1}$ and added to the prediction.

10 The Extended Kalman Filter *(EKF deck, slides 22–24)*

Consider a *nonlinear* discrete-time system with noise

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k) + \boldsymbol{v}_k, \qquad \boldsymbol{y}_k = \boldsymbol{h}_k(\boldsymbol{x}_k) + \boldsymbol{w}_k, \tag{18}$$

where $\boldsymbol{f}_k$ and $\boldsymbol{h}_k$ are C^1 functions. One simple way to build a filter: *linearize* the equations around the current estimate and apply the KF equations to the linear approximation. However, the nonlinear model is still used for the prediction, $\hat{\boldsymbol{x}}_{k+1|k} = \boldsymbol{f}_k(\hat{\boldsymbol{x}}_k, \boldsymbol{u}_k)$, and for the innovation, $\boldsymbol{\nu}_{k+1} = \boldsymbol{y}_{k+1} - \boldsymbol{h}_{k+1}(\hat{\boldsymbol{x}}_{k+1|k})$.

Algorithm 10.1 (Extended Kalman Filter).

$$\begin{aligned} \hat{\boldsymbol{x}}_{k+1|k} &= \boldsymbol{f}_k(\hat{\boldsymbol{x}}_k, \boldsymbol{u}_k), \\ \mathbf{P}_{k+1|k} &= \mathbf{F}_k \mathbf{P}_k \mathbf{F}_k^\top + \mathbf{V}_k, \\ \hat{\boldsymbol{x}}_{k+1} &= \hat{\boldsymbol{x}}_{k+1|k} + \mathbf{R}_{k+1} \boldsymbol{\nu}_{k+1}, \\ \mathbf{P}_{k+1} &= \mathbf{P}_{k+1|k} - \mathbf{R}_{k+1} \mathbf{H}_{k+1} \mathbf{P}_{k+1|k}, \end{aligned} \tag{19}$$

with the Jacobians and gain

$$\mathbf{F}_k = \frac{\partial \boldsymbol{f}_k}{\partial \boldsymbol{x}}\Big|_{\boldsymbol{x}=\hat{\boldsymbol{x}}_k}, \qquad \mathbf{H}_{k+1} = \frac{\partial \boldsymbol{h}_{k+1}}{\partial \boldsymbol{x}}\Big|_{\boldsymbol{x}=\hat{\boldsymbol{x}}_{k+1|k}},$$
$$\mathbf{R}_{k+1} = \mathbf{P}_{k+1|k} \mathbf{H}_{k+1}^\top \big(\mathbf{H}_{k+1} \mathbf{P}_{k+1|k} \mathbf{H}_{k+1}^\top + \mathbf{W}_{k+1}\big)^{-1}.$$

Remarks. Prediction and innovation come from the nonlinear equations, while the covariance is propagated using their *linearizations*. Unlike the KF for linear-Gaussian systems, the EKF is *not* guaranteed to be optimal: even with Gaussian initial uncertainty and noises, the nonlinearity makes the state distribution non-Gaussian, and the EKF approximates it by a Gaussian. Accuracy depends on the validity of the linearization, hence on the size of the estimation uncertainty and on the system nonlinearities.

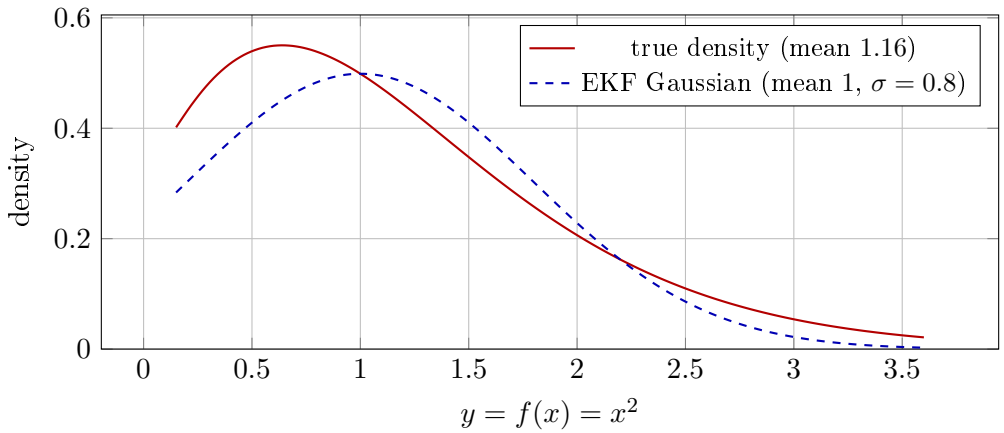


Figure 10: Why the EKF is only approximate (illustrative toy example, original): $x \sim \mathcal{N}(1, 0.4^2)$ is passed through $f(x) = x^2$. The true output density is skewed with mean $\mu^2 + \sigma^2 = 1.16$; the EKF replaces it with a Gaussian centered at $f(\mu) = 1$ and with standard deviation $|f'(\mu)|\sigma = 0.8$.

Intuition

Linearizing is like replacing a curved road by its tangent line at the current estimate: fine as long as the uncertainty cloud is small compared with the curvature, misleading when it is large. This is why an EKF that has become overconfident or has a very large initial uncertainty can fail, and why the quality of the filter hinges on how good the estimate already is.

11 EKF localization of a unicycle with range-and-bearing landmarks *(book, Sect. 7.7.2)*

We now apply the EKF to a unicycle equipped with a sensor that measures *range* (relative distance) and *bearing* to *landmarks*: easily recognizable elements of the environment, natural (features, objects) or artificial (devices added for localization), whose positions are known to the robot. Depending on the robot configuration and the field of view, only a subset of the landmarks is visible (Fig. 11).

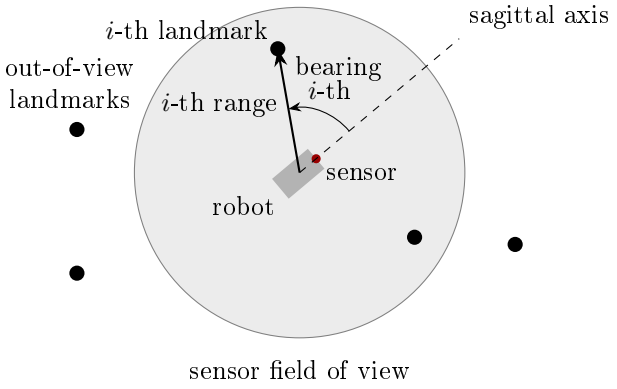


Figure 11: A typical setting for EKF-based localization with range and bearing measurements to known landmarks (redrawn). The bearing is measured from the robot's sagittal axis; the landmarks outside the field of view do not produce measurements.

11.1 Motion model

Any of the odometric models (2), (3) or (4) can serve as the discrete-time motion model, with v_k, ω_k reconstructed from the encoders as in Sect. 3. Taking the Euler one and adding *motion noise* $\boldsymbol{v}_k = (v_{1,k}, v_{2,k}, v_{3,k})$ (white, Gaussian, zero mean, covariance $\mathbf{V}_k$):

$$\begin{aligned} x_{k+1} &= x_k + v_k T_s \cos\theta_k + v_{1,k}, \\ y_{k+1} &= y_k + v_k T_s \sin\theta_k + v_{2,k}, \\ \theta_{k+1} &= \theta_k + \omega_k T_s + v_{3,k}. \end{aligned} \tag{20}$$

11.2 Measurement model

Let L be the number of landmarks, $\boldsymbol{p}_{l,i} = (x_{l,i}, y_{l,i})$ the known position of the i-th one, and L_k the number of landmarks actually seen at t_k. Assume the identity of each observed landmark is known (landmarks are tagged, e.g. by shape or color), which allows building the *association map*

$$a : \{1, \ldots, L_k\} \to \{1, \ldots, L\},$$

where $a(i)$ is the identity of the i-th observed landmark. Assuming the sensor sits at the center of the wheel axle, the measurement model is

$$\boldsymbol{y}_k = \begin{pmatrix} \boldsymbol{h}_1(\boldsymbol{q}_k, a(1)) \\ \vdots \\ \boldsymbol{h}_{L_k}(\boldsymbol{q}_k, a(L_k)) \end{pmatrix} + \begin{pmatrix} \boldsymbol{w}_{1,k} \\ \vdots \\ \boldsymbol{w}_{L_k,k} \end{pmatrix}, \qquad \boldsymbol{h}_i(\boldsymbol{q}_k, a(i)) = \begin{pmatrix} \sqrt{(x_k - x_{l,a(i)})^2 + (y_k - y_{l,a(i)})^2} \\ \operatorname{atan2}(y_{l,a(i)} - y_k,\ x_{l,a(i)} - x_k) - \theta_k \end{pmatrix}, \tag{21}$$

with $\boldsymbol{w}_k$ white, Gaussian, zero mean, covariance $\mathbf{W}_k$. Both the motion and the measurement model are nonlinear.

11.3 The EKF equations for this problem

Prediction.

$$\hat{\boldsymbol{q}}_{k+1|k} = \begin{pmatrix} \hat{x}_{k+1|k} \\ \hat{y}_{k+1|k} \\ \hat{\theta}_{k+1|k} \end{pmatrix} = \begin{pmatrix} \hat{x}_k + v_k T_s \cos\hat{\theta}_k \\ \hat{y}_k + v_k T_s \sin\hat{\theta}_k \\ \hat{\theta}_k + \omega_k T_s \end{pmatrix}, \qquad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_k \mathbf{F}_k^\top + \mathbf{V}_k,$$

$$\mathbf{F}_k = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{q}_k}\Big|_{\hat{\boldsymbol{q}}_k} = \begin{pmatrix} 1 & 0 & -v_k T_s \sin\hat{\theta}_k \\ 0 & 1 & v_k T_s \cos\hat{\theta}_k \\ 0 & 0 & 1 \end{pmatrix}. \tag{22}$$

Correction. $\hat{\boldsymbol{q}}_{k+1} = \hat{\boldsymbol{q}}_{k+1|k} + \mathbf{R}_{k+1}\boldsymbol{\nu}_{k+1}$, $\mathbf{P}_{k+1} = \mathbf{P}_{k+1|k} - \mathbf{R}_{k+1}\mathbf{H}_{k+1}\mathbf{P}_{k+1|k}$, with $\mathbf{R}_{k+1} = \mathbf{P}_{k+1|k}\mathbf{H}_{k+1}^\top(\mathbf{H}_{k+1}\mathbf{P}_{k+1|k}\mathbf{H}_{k+1}^\top + \mathbf{W}_{k+1})^{-1}$ and the innovation $\boldsymbol{\nu}_{k+1} = \boldsymbol{y}_{k+1} - \boldsymbol{h}_{k+1}(\hat{\boldsymbol{q}}_{k+1|k})$ (actual minus predicted measurements). The matrix $\mathbf{H}_{k+1}$ stacks the L_k blocks

$$\frac{\partial \boldsymbol{h}_i}{\partial \boldsymbol{q}_k}\Big|_{\hat{\boldsymbol{q}}_{k+1|k}} = \begin{pmatrix} \dfrac{\hat{x}_{k+1|k} - x_{l,a(i)}}{\rho_i} & \dfrac{\hat{y}_{k+1|k} - y_{l,a(i)}}{\rho_i} & 0 \\ -\dfrac{\hat{y}_{k+1|k} - y_{l,a(i)}}{\rho_i^2} & \dfrac{\hat{x}_{k+1|k} - x_{l,a(i)}}{\rho_i^2} & -1 \end{pmatrix}, \qquad \rho_i = \|\boldsymbol{p}_{l,a(i)} - \hat{\boldsymbol{p}}_{k+1|k}\|, \tag{23}$$

$i = 1, \ldots, L_k$, where $\hat{\boldsymbol{p}}_{k+1|k} = (\hat{x}_{k+1|k}, \hat{y}_{k+1|k})$.

Remark. In the bearing row of (23) the denominators are ρ_i^2. This follows from differentiating $\operatorname{atan2}(Y, X)$, whose partial derivatives are $-Y/(X^2+Y^2)$ and $X/(X^2+Y^2)$, with $X = x_l - x_k$, $Y = y_l - y_k$.

Intuition

Reading $\mathbf{F}_k$. The third column says that a heading error $\delta\theta$ moves the predicted position sideways by $v_k T_s\,\delta\theta$ per step: heading uncertainty is turned into position uncertainty, and the longer the robot drives, the more it accumulates (exactly the drift mechanism of Sect. 4, now quantified inside $\mathbf{P}$).

Reading $\mathbf{H}$. The first row is the unit vector pointing from the landmark to the robot: moving the robot directly away from the landmark increases the range at rate 1, moving sideways does not change it (to first order). *Worked numbers:* predicted pose $(0, 0, 0)$, landmark at $(3, 4)$, so $\rho = 5$: predicted range 5, predicted bearing $\operatorname{atan2}(4, 3) - 0 = 0.927$ rad, and $\mathbf{H}_i = \left(\begin{smallmatrix} -0.6 & -0.8 & 0 \\ 0.16 & -0.12 & -1 \end{smallmatrix}\right)$. Moving the robot 1 m toward the landmark along x shortens the range by about 0.6 m (first entry -0.6); a heading error of $+0.1$ rad shifts the bearing by -0.1 rad (last entry -1). A single landmark gives two scalar equations for three unknowns (x, y, θ), so generally at least two landmarks are needed to constrain the full pose from one set of readings; the stacked $\mathbf{H}$ is what combines them.

Integration scheme	**Update of x_{k+1} (and y)**	**Local error**	**Comment**
Euler	$x_k + v_k T_s \cos\theta_k$	$\approx \frac{1}{2} v\omega T_s^2$	cheapest; uses the initial heading
Runge–Kutta (2nd order)	$x_k + v_k T_s \cos(\theta_k + \frac{\omega_k T_s}{2})$	$\approx \frac{1}{24} v\omega^2 T_s^3$	uses the mean heading; no division
Exact	$x_k + \frac{v_k}{\omega_k}(\sin\theta_{k+1} - \sin\theta_k)$	none (constant inputs)	needs a conditional for $\omega_k \approx 0$

Table 1: Odometric integration schemes (all have exact $\theta_{k+1} = \theta_k + \omega_k T_s$).

Estimator	**Model / sensors**	**What it provides**	**Main limitation**
Dead reckoning (odometry)	nonlinear kinematics; proprioceptive only	$\hat{\boldsymbol{q}}$ only; exact only if $\boldsymbol{q}_0$, model and encoders are perfect	unbounded drift
Naive observer (no noise)	linear; exact output $\boldsymbol{y}$	$\hat{\boldsymbol{x}}$ via $\mathbf{C}^{\dagger}\boldsymbol{\nu}$ (min-norm)	errors along Ω not corrected; ignores uncertainty
KF, process noise only	linear; exact output	$\hat{\boldsymbol{x}}, \mathbf{P}$ via weighted $\mathbf{C}_M^{\dagger}$	$\mathbf{P}$ becomes singular
Kalman filter	linear; noisy output, Gaussian noises	$\hat{\boldsymbol{x}}, \mathbf{P}$ with gain $\mathbf{R}$; MMSE, correct posterior	needs linear-Gaussian setting
Extended Kalman filter	nonlinear C^1 $\boldsymbol{f}, \boldsymbol{h}$; noisy exteroceptive data (e.g. landmarks)	$\hat{\boldsymbol{x}}, \mathbf{P}$; drift removed	approximate (linearization, Gaussian assumption); no optimality guarantee

Table 2: From dead reckoning to the EKF.

12 Summary

12.1 Comparison of methods

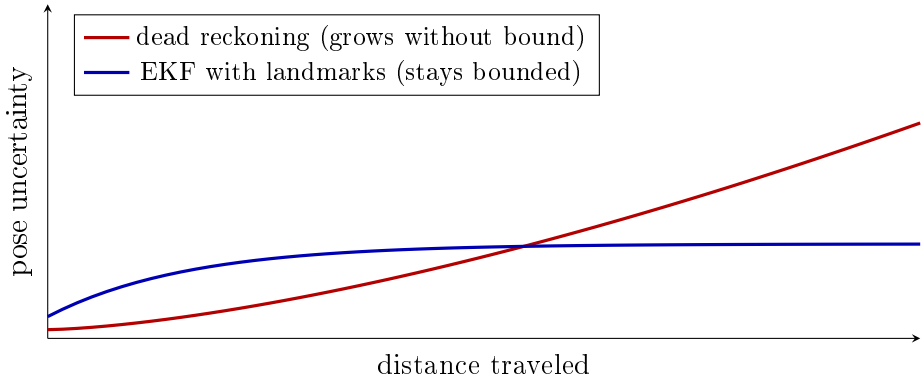


Figure 12: Qualitative sketch (not measured data): pure odometry accumulates uncertainty with the distance traveled, while the EKF alternates growth during prediction with reductions at every correction, so the uncertainty settles at a bounded level.

12.2 Key take-aways

- **Odometry is integration.** Under piecewise-constant inputs the unicycle moves on arcs; Euler, Runge–Kutta and exact formulas differ only in how well they follow the arc (error per step $\propto T_s^2$, T_s^3, none), and the inputs should be reconstructed from encoders $(\Delta s, \Delta\theta)$, not taken from the commands.
- **Dead reckoning always drifts.** Slippage, calibration errors and encoder noise are integrated, and small wheel errors become large heading errors through the small wheelbase; accuracy can never be better than that of $\boldsymbol{q}_0$.
- **Predict, then correct.** The Kalman filter propagates a Gaussian (mean and covariance) through the model, then pulls the mean toward the measurement along the direction the filter is least sure about, in the sense of the Mahalanobis distance.
- **The gain $\mathbf{R} = \mathbf{P}\mathbf{C}^\top(\mathbf{C}\mathbf{P}\mathbf{C}^\top + \mathbf{W})^{-1}$ balances prediction and sensor.** Large $\mathbf{R}$: trust the measurements; small $\mathbf{R}$: trust the model. The filter is MMSE and correct for linear-Gaussian systems.
- **The EKF linearizes.** It keeps the nonlinear model for prediction and innovation and uses Jacobians for the covariance; this is what makes range-and-bearing localization with known landmarks workable, at the price of approximation and no optimality guarantee.